Computer Vision Mini Project Report

Jasser Abdelfattah
UH ID: 21033101

Problem Statement and Dataset Description

Object recognition is a critical computer vision task that involves identifying objects within images. The CIFAR-10 dataset serves as a benchmark for this mini-project, consisting of 60,000 32×32 color images distributed equally across ten classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each class contains 6,000 images, with 50,000 allocated for training and 10,000 for testing.

The primary objective of this project is to develop and evaluate an object recognition model capable of accurately classifying images into these ten categories. This system could have applications in educational tools for nursery students to learn object categories.

Approaches Used

First Pipeline (`custom_model_cifar10_test.ipynb`)

In this pipeline, histogram equalization was applied as a preprocessing step to enhance image contrast and normalize the CIFAR-10 dataset. The processed data was then used to train a custom Convolutional Neural Network (CNN) with the following architecture:

- **Convolutional Layers:**
  - `conv1`: $3 \rightarrow 16$ filters, kernel size: 3×3, stride: 1, padding: 1
  - `conv2`: $16 \rightarrow 32$ filters, kernel size: 3×3, stride: 1, padding: 1
  - `conv3`: $32 \rightarrow 64$ filters, kernel size: 3×3, stride: 1, padding: 1
- **Fully Connected Layers:**
  - `fc1`: $1024 \rightarrow 500$ neurons, activated by ReLU.

- `fc2`: 500 → 10 neurons (final classification layer).

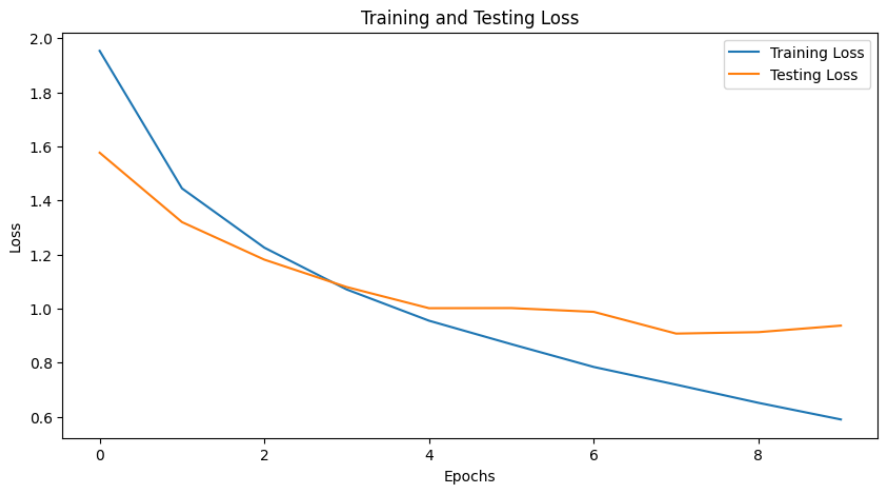
- **Regularization:** Dropout with $p = 0.2$ to reduce overfitting.

The model was trained on a Tesla T4 GPU using Google Colab, with the following hyperparameters:

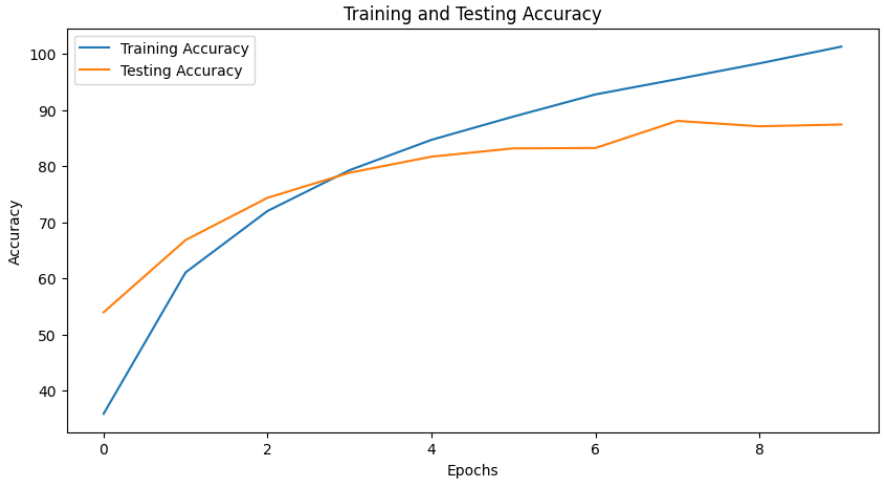
- **Loss Function:** Cross-Entropy Loss
- **Optimizer:** Stochastic Gradient Descent (SGD)
- **Learning Rate:** 0.00974
- **Number of Epochs:** 10
- **Batch Size:** 128
- CNN Architecture:

```
CCN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=1024, out_features=500, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
  (fc2): Linear(in_features=500, out_features=10, bias=True)
)
```

- Training and Testing Loss:



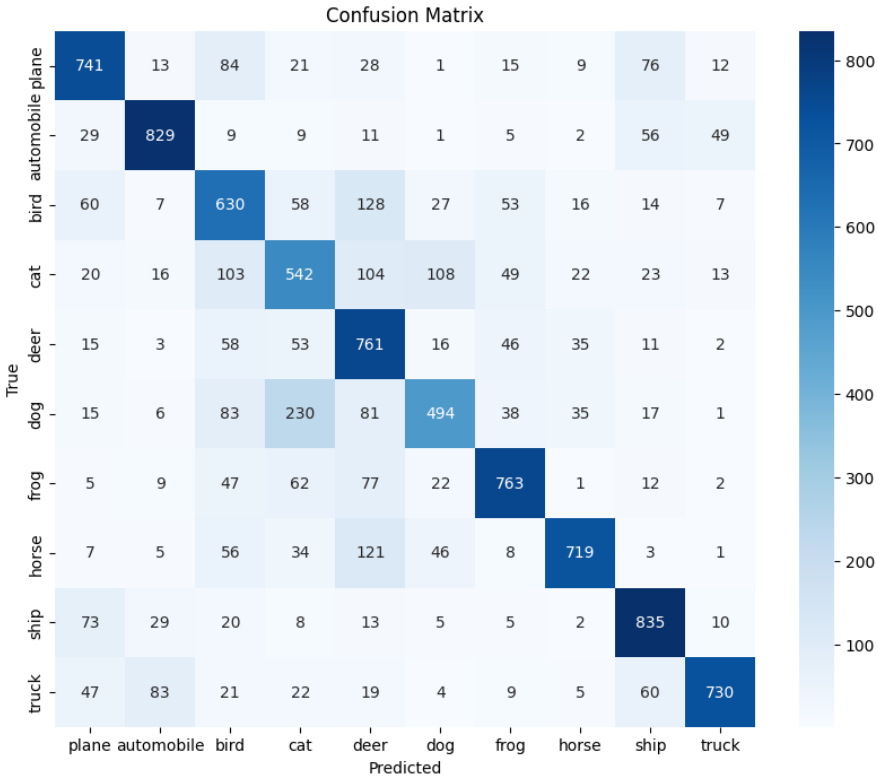
- Training and Testing Accuracy:



- Classification Report:

	precision	recall	f1-score	support
plane	0.73	0.74	0.74	1000
automobile	0.83	0.83	0.83	1000
bird	0.57	0.63	0.60	1000
cat	0.52	0.54	0.53	1000
deer	0.57	0.76	0.65	1000
dog	0.68	0.49	0.57	1000
frog	0.77	0.76	0.77	1000
horse	0.85	0.72	0.78	1000
ship	0.75	0.83	0.79	1000
truck	0.88	0.73	0.80	1000
accuracy			0.70	10000
macro avg	0.72	0.70	0.71	10000
weighted avg	0.72	0.70	0.71	10000

- Confusion Matrix:



- Accuracy on the 10,000 Test Images:

```
Accuracy of plane : 74 %
Accuracy of automobile : 82 %
Accuracy of  bird : 63 %
Accuracy of   cat : 54 %
Accuracy of  deer : 76 %
Accuracy of   dog : 49 %
Accuracy of  frog : 76 %
Accuracy of horse : 71 %
Accuracy of  ship : 83 %
Accuracy of truck : 73 %
Accuracy of the network on the 10000 test images: 70 %
```

Second Pipeline (`resnet_cifar10_test.ipynb`)

In this pipeline, only normalization was applied to the CIFAR-10 dataset as preprocessing since the ResNet model demonstrated robust performance without additional enhancements. A pre-trained ResNet model was fine-tuned for the task,

trained for 10 epochs on the same Tesla T4 GPU. The training utilized the following configurations:

- **Loss Function:** Cross-Entropy Loss
- **Optimizer:** Adam
- **Learning Rate:** 0.01
- **Batch Size:** 128
- ResNet Architecture:

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
...
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=10, bias=True)
)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

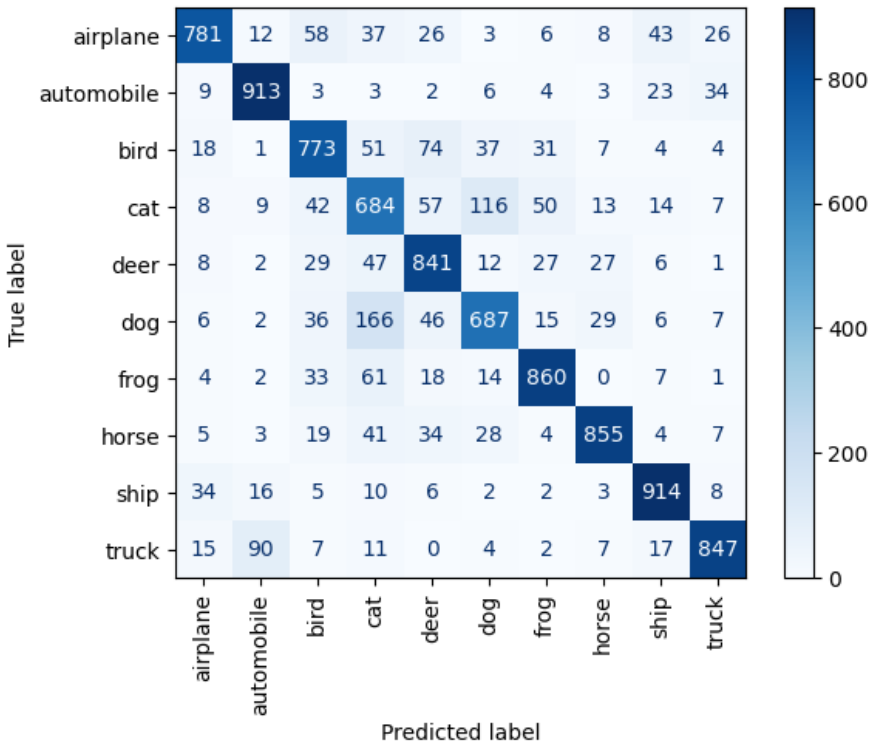
- Classification Report:

```
Classification Report:
              precision    recall  f1-score   support

    airplane       0.88      0.78      0.83      1000
  automobile       0.87      0.91      0.89      1000
        bird       0.77      0.77      0.77      1000
         cat       0.62      0.68      0.65      1000
        deer       0.76      0.84      0.80      1000
         dog       0.76      0.69      0.72      1000
        frog       0.86      0.86      0.86      1000
       horse       0.90      0.85      0.88      1000
        ship       0.88      0.91      0.90      1000
       truck       0.90      0.85      0.87      1000

    accuracy                           0.82     10000
   macro avg       0.82      0.82      0.82     10000
weighted avg       0.82      0.82      0.82     10000
```

- Confusion Matrix:



- Accuracy on the 10,000 Test Images:

```
Accuracy of plane : 79 %
Accuracy of automobile : 90 %
Accuracy of  bird : 75 %
Accuracy of   cat : 66 %
Accuracy of  deer : 80 %
Accuracy of   dog : 70 %
Accuracy of  frog : 85 %
Accuracy of horse : 84 %
Accuracy of  ship : 89 %
Accuracy of truck : 84 %
Accuracy of the network on the 10000 test images: 80 %
```

Model Performance on Unseen Data

Unseen Image Classes	**Custom_CCN_Model Predicted**	**ResNet Predicted**
Plane	Plane	Plane
Truck	Truck	Truck
Ship	Ship	Ship
Frog	Frog	Frog
Automobile	Automobile	Automobile
Deer	Deer	Deer
Horse	Horse	Horse
Cat	Cat	Ship
Bird	Bird	Bird
Dog	Deer	Horse

Table 1: Predictions for Unseen Image Classes by Custom CCN Model and ResNet.

Key Observations

Accurate Predictions: Both models correctly predicted several classes such as **Plane**, **Truck**, **Ship**, **Frog**, **Automobile**, **Deer**, **Horse**, and **Bird**.

Misclassifications:

- **The Custom_CCN_Model:**
  - Misclassified **Dog** as **Deer**.
- **The ResNet:**
  - Misclassified **Cat** as **Ship**.
  - Misclassified **Dog** as **Horse**.

Discussion of Results

The results from both pipelines highlight the strengths and limitations of the two models in the context of object recognition using the CIFAR-10 dataset.

Custom_CCN_Model: The Custom CNN model achieved consistent performance across most classes, correctly predicting nine out of ten unseen image classes. Its architecture, specifically designed for the CIFAR-10 dataset, demonstrates its ability to generalize effectively to new data. However, the model misclassified the **Dog** class as **Deer**, indicating that these classes may share visual features that confuse the network. This could be due to insufficient feature differentiation in the fully connected layers or the need for additional data augmentation during training to enhance its robustness.

ResNet: The ResNet model, leveraging pre-trained weights, also performed well, accurately predicting eight out of ten classes. However, it misclassified **Cat** as **Ship** and **Dog** as **Horse**. These errors suggest that while ResNet is highly effective for general classification tasks, its fine-tuning on CIFAR-10 might not have fully captured the nuances of certain class boundaries. This could be addressed by adjusting the learning rate or extending the fine-tuning process with a focus on class-specific examples.

Comparison: Both models demonstrate strong overall performance, with each correctly predicting the same number of unseen classes. The Custom CNN model shows promise in scenarios where a lightweight, task-specific architecture is preferred. ResNet, on the other hand, benefits from the transfer learning approach, which provides a more generalized feature extraction capability.

The misclassifications highlight the need for further investigation into the following areas:

- Enhancing the feature representation of visually similar classes (e.g., **Dog** vs. **Deer**, **Cat** vs. **Ship**).
- Experimenting with additional preprocessing techniques such as advanced augmentation or adaptive histogram equalization.
- Optimizing training strategies, such as class-weighted loss functions or curriculum learning, to improve performance on challenging classes.

Future Work: To improve the robustness of both models, future work could involve:

- Increasing the diversity of training data through synthetic augmentation techniques.
- Investigating ensemble methods that combine predictions from both Custom CNN and ResNet for better overall accuracy.
- Exploring alternative architectures, such as Vision Transformers, to assess their performance on the same dataset.

Overall, the results underscore the effectiveness of both approaches, while emphasizing the potential for further enhancements to tackle edge cases in classification.